

ELEC/COEN 390

Engineering team design project

MD NAZMUL HOQ

KHADIZA SARWAR MOURY

CONCORDIA UNIVERSITY

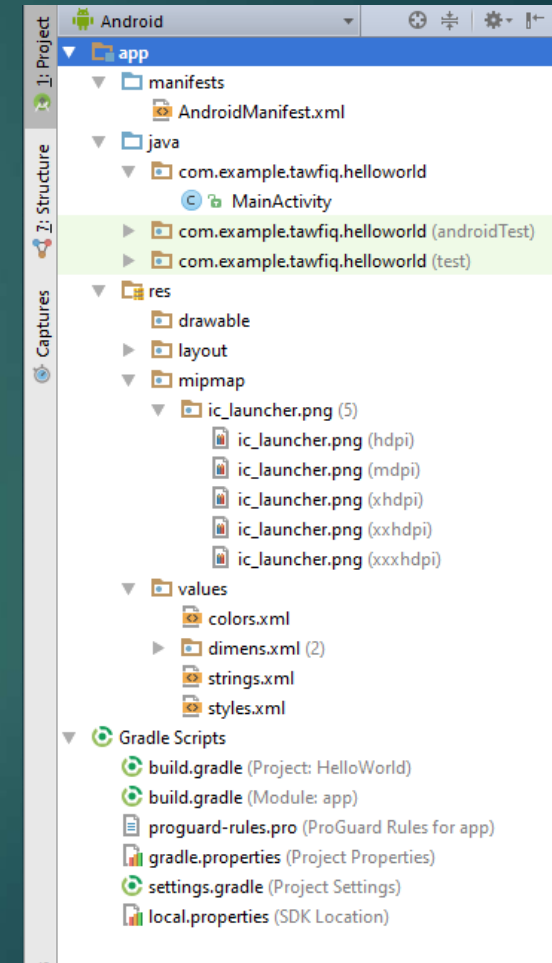
Outline

- ▶ Structure of an Android project in Android Studio
- ▶ User Interface
- ▶ Activity life cycle
- ▶ Intent
- ▶ Hello World App

Structure of an Android Project

3

- ▶ **Manifest file:**
 - ▶ Presents the App's organization & lists components
- ▶ **Java src code:**
 - ▶ Java classes
- ▶ **Resources:**
 - ▶ Images
 - ▶ Icons
 - ▶ common dimensions/strings etc.
- ▶ **Gradle Scripts:**
 - ▶ Build scripts
 - ▶ where to include 3rd party libs



AndroidManifest.xml File

4

- ▶ **Application:**

- ▶ Defines the application

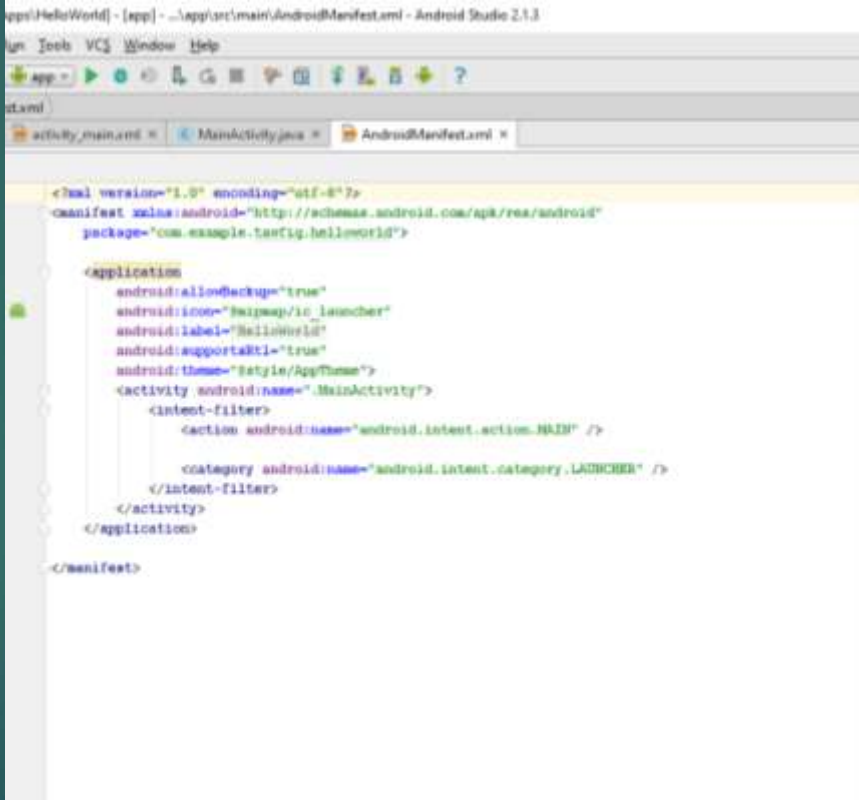
- ▶ **Activity:**

- ▶ Defines a particular activity

- ▶ **Intent Filters:**

- ▶ Specifies the type of intents that your activity can be started with.

This manifest file has three main components - **application**, **activity**, and **intent**. We can see that activity “MainActivity” can handle the *android.intent.action.MAIN* intent. This is the “call” that android makes to your application asking it to present its main activity when the application starts.



```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.taofig.helloworld">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/hello_world"
        android:supportRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

AndroidManifest.xml File

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.coen390.amine.randomfacts">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="Random Facts"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

- ▶ The manifest file itself tells Android what's needed before your app can be executed.
- ▶ For more info you can go here:
<http://developer.android.com/guide/topics/manifest/uses-permission-element.html>

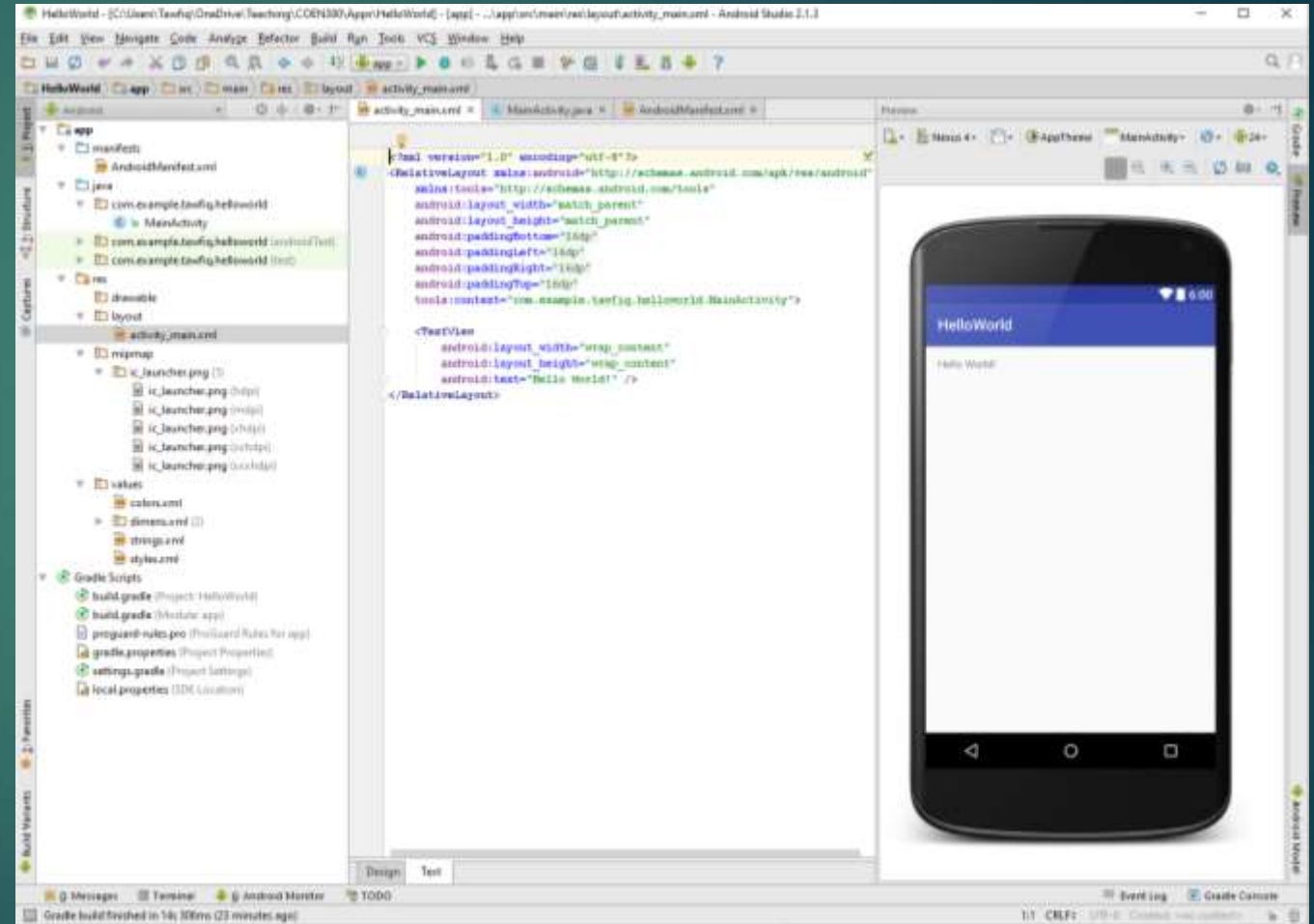
Activity

- ▶ An activity represents a single screen with a user interface just like window or frame of Java.Android.
- ▶ Activity is the subclass of ContextThemeWrapper class.
- ▶ Activity has a:
 - ▶ Layout: View hierarchies that control screen format and appearance of the views (the UI).
 - ▶ Java class

Activity Layout

7

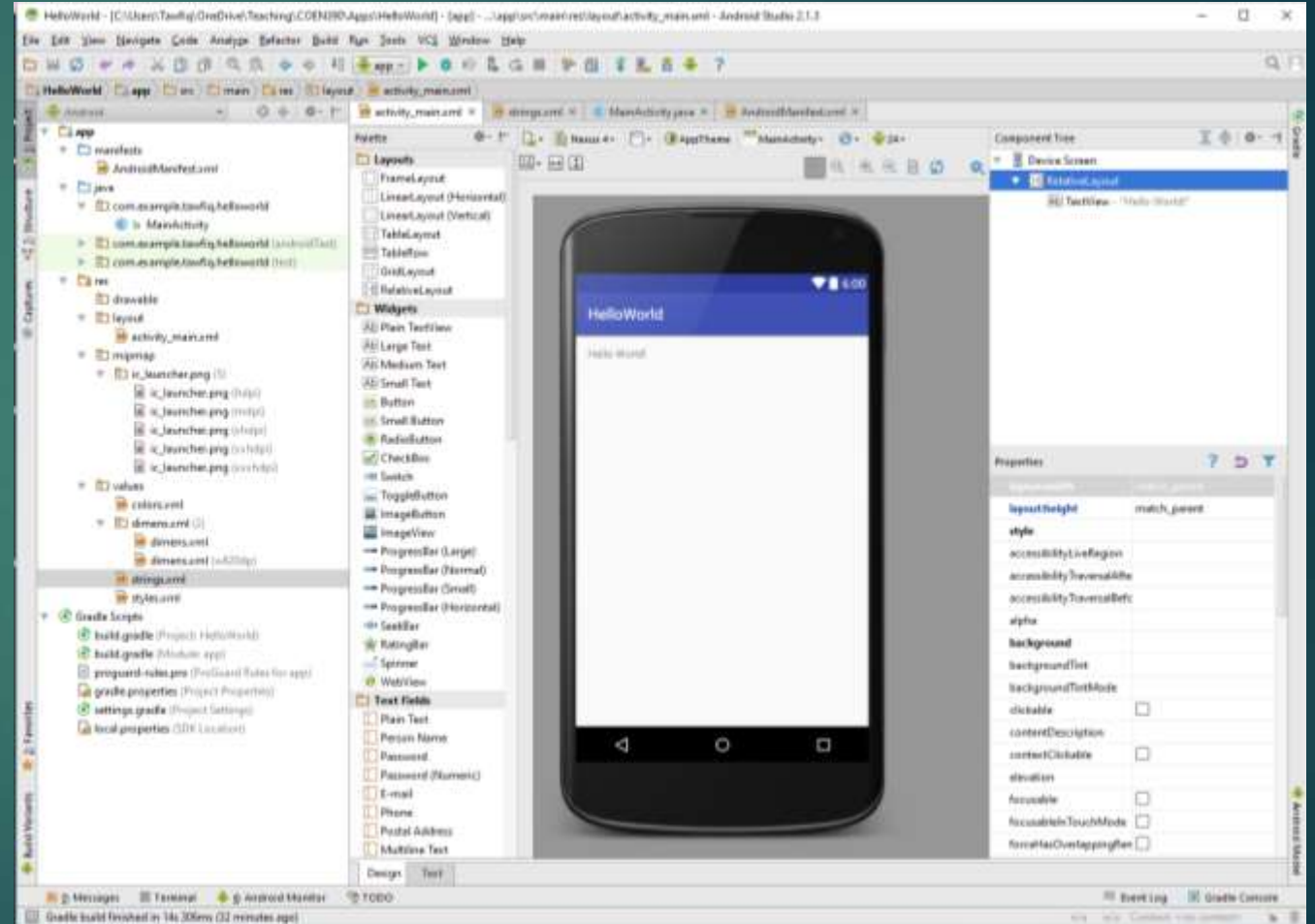
- ▶ The layout is an xml file
- ▶ Can be viewed as text or in Design mode that Android Studio provide
- ▶ Use resource files:
 - ▶ strings.xml
 - ▶ styles.xml



Use Design Mode

8

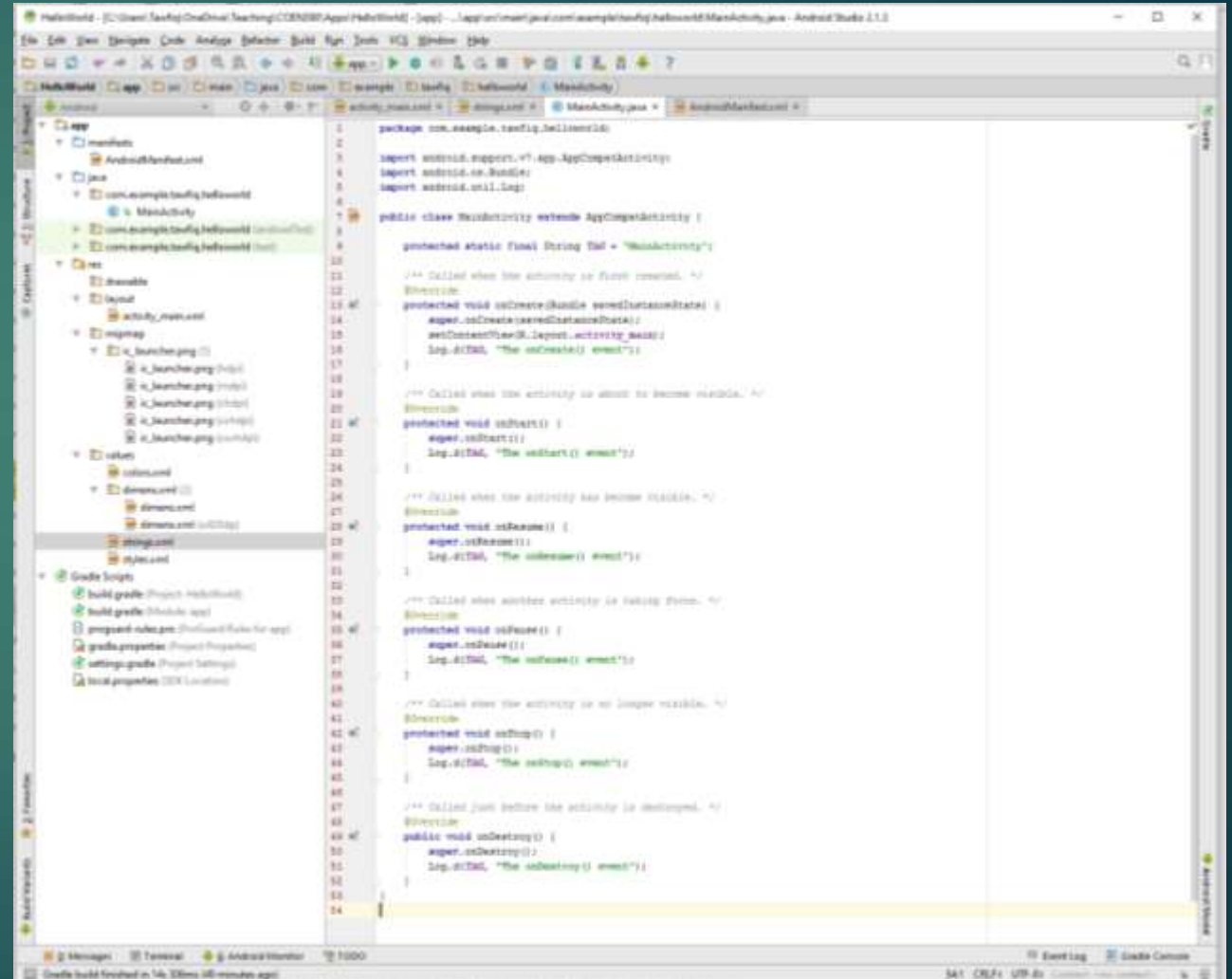
- ▶ It's easier.
- ▶ Drag and Drop
- ▶ Changing properties will automatically edit the xml file
- ▶ “Once you provide a resource in your application (discussed in [Providing Resources](#)), you can apply it by referencing its resource ID. All resource IDs are defined in your project's R class, which the aapt tool automatically generates.”

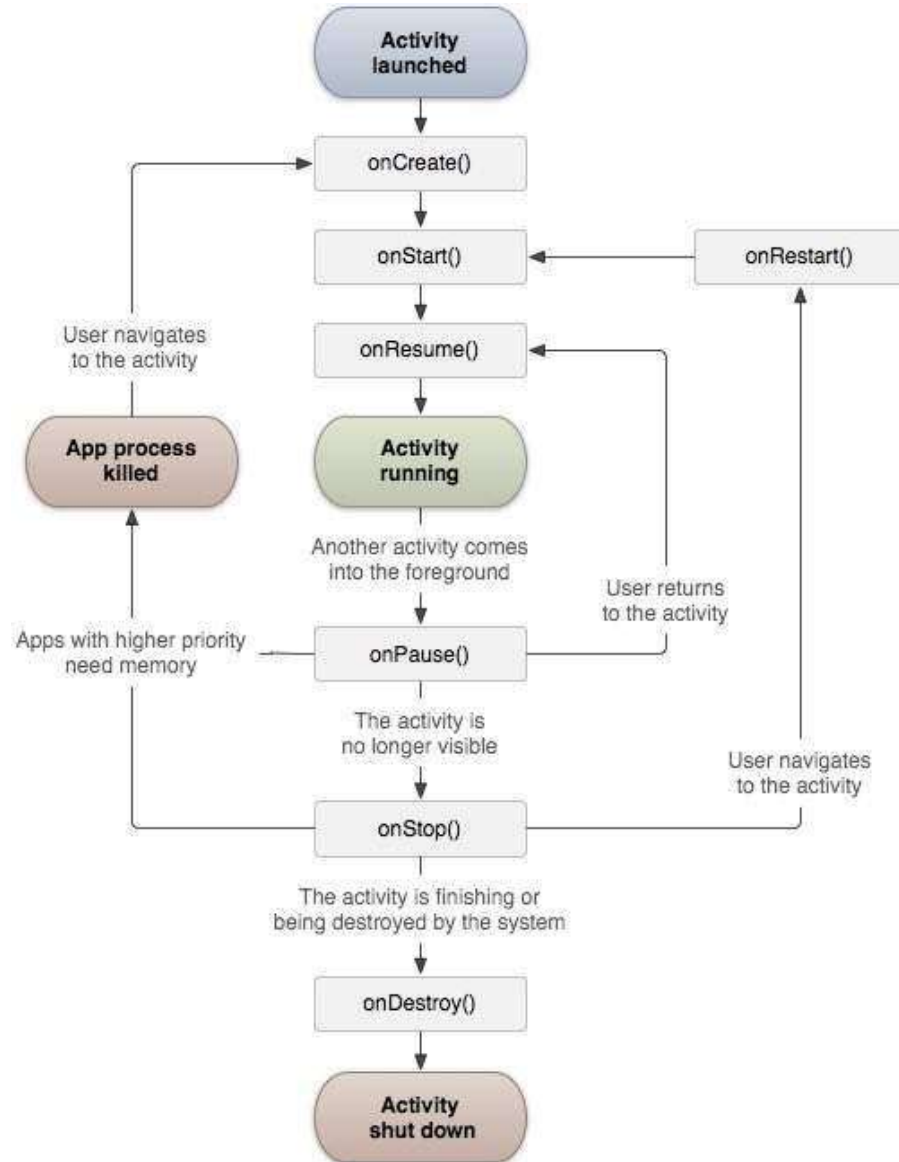


Activity Java Class

9

- ▶ Inheritance: our MainActivity class takes all attributes and functionality of AppCompatActivity/Activity
- ▶ OnCreate Function:
 - ▶ Initialize the Activity.
 - ▶ Do all Setup.
 - ▶ Get access to elements as needed.
 - ▶ Render the page's UI using setContentView.



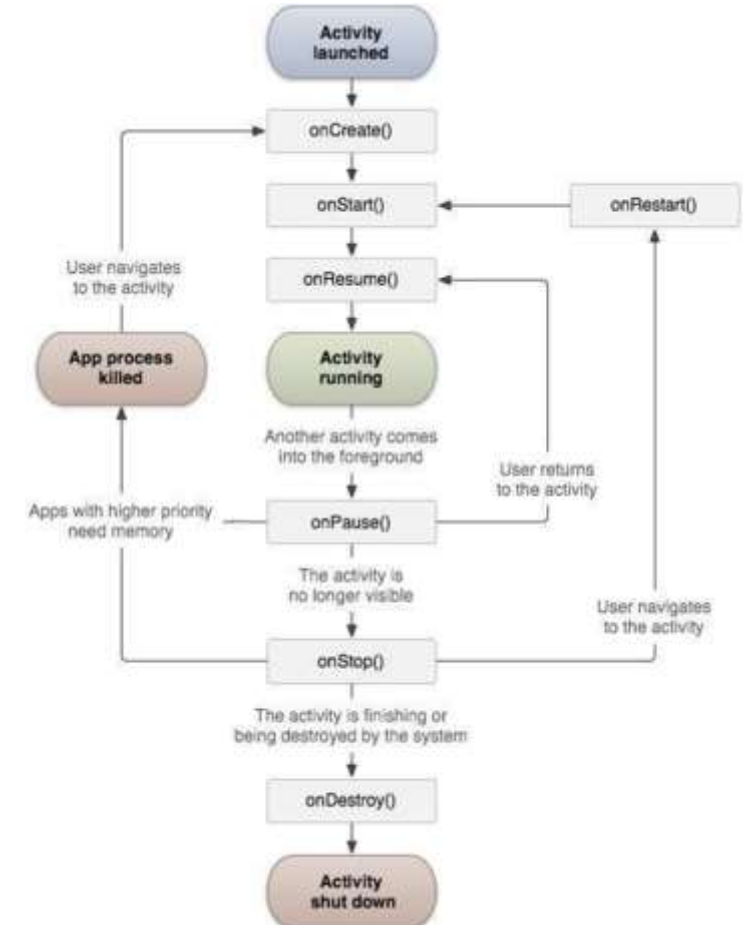


Activity's Life Cycle

Activity's Life Cycle

11

- ▶ **onStart()**: Called before UI shows up to the user. Start UI animations, audio based content or anything else that requires the activity's contents to be on screen.
- ▶ **onResume()**: Called before a method is brought to the foreground.
Restart animations, Update UI elements or initialize any components that you
- ▶ **onPause()**: gets called before sliding into the background (whether a dialog is open or if it's going to)
You stop any visuals or audio associated with the activity.
- ▶ **onStop()** called after onPause(), but before the activity goes into the background.
You save data that you want to commit to the disk.



Source:
http://www.tutorialspoint.com/android/android_activities.htm

Intent

12

- ▶ Based on [developer.android documentation](#):
 - ▶ It is an abstract description of an operation to be performed.
 - ▶ can be commonly thought of as glue between activities.
- ▶ It has other uses such as:
 - ▶ Starting a service.
 - ▶ Being sent to BroadcastReceiver Components.

Intent Forms

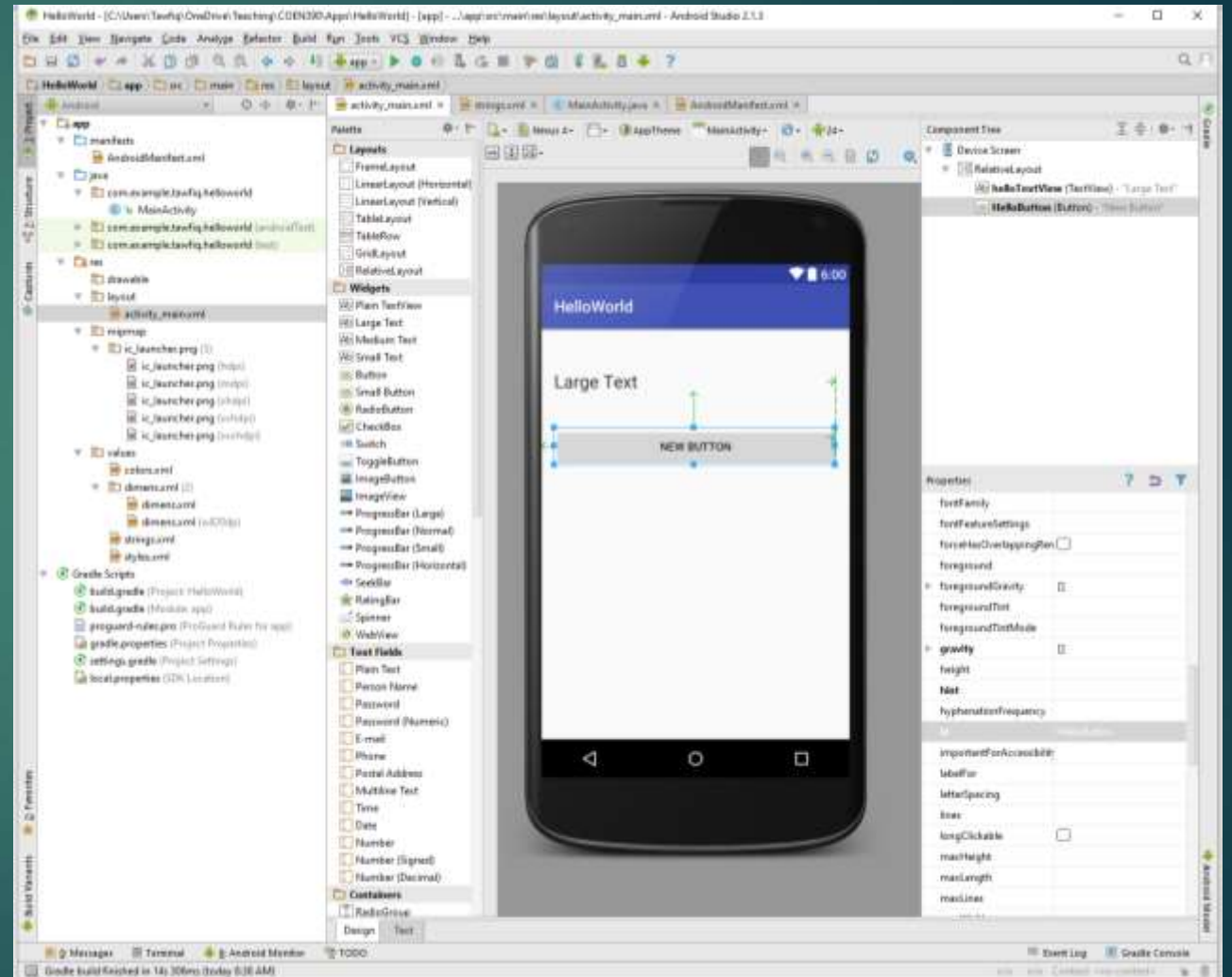
13

- ▶ **Explicit Intents:** has a specified component set, which provides the exact class to run so no other info is needed.
 - ▶ It is a way for an application to launch various internal activities it has as the user interacts with the app.
 - ▶ Example: use intent to go to another activity **startActivity(intent)**
- ▶ **Implicit Intents:** “do not name a specific component, but instead declare a general action to perform, which allows a component from another app to handle it. For example, if you want to show the user a **location on a map**, you can use an implicit intent to request that another capable app show a specified location on a map.”
- ▶ More on intents and examples:
<https://developer.android.com/guide/components/intents-filters.html>

Hello World

14

- ▶ Drag and drop UI elements
- ▶ Edit position and size from Design mode or Text mode. Design is always easier
- ▶ Change the ids in the Properties (on the right)
- ▶ Android Studio will automatically update the android project files like the R class and layout xml file



Connect the UI elements to the Activity's Java class

15

```
example tawfiq helloworld MainActivity
activity_main.xml x strings.xml x MainActivity.java x AndroidManifest.xml x

7 import android.widget.Button;
8 import android.widget.TextView;
9
10 public class MainActivity extends AppCompatActivity {
11
12     protected static final String TAG = "MainActivity";
13
14     /**define the UI elements you will use in your code**/
15     protected Button helloButton = null;
16     protected TextView helloTextView;
17
18     /** Called when the activity is first created. */
19     @Override
20     protected void onCreate(Bundle savedInstanceState) {
21         super.onCreate(savedInstanceState);
22         setContentView(R.layout.activity_main);
23         Log.d(TAG, "The onCreate() event");
24
25         setupUI();
26     }
27
28     protected void setupUI()
29     {
30         helloTextView = (TextView) findViewById(R.id.helloTextView);
31         helloButton = (Button) findViewById(R.id.helloButton);
32
33         helloButton.setOnClickListener(onClickHelloButton);
34     }
35
36     /** Called when the activity is about to become visible. */
37     @Override
38     protected void onStart() {...}
39
40
41
42
43
44
45
46
```

```
example tawfiq helloworld MainActivity
activity_main.xml x strings.xml x MainActivity.java x AndroidManifest.xml x

61 /** Called when the activity is no longer visible. */
62 @Override
63 protected void onStop() {...}
64
65 /** Called just before the activity is destroyed. */
66 @Override
67 public void onDestroy() {
68     super.onDestroy();
69     Log.d(TAG, "The onDestroy() event");
70 }
71
72 /**
73  * button onclick listeners
74  */
75 private Button.OnClickListener onClickHelloButton = new Button.OnClickListener() {
76
77     @Override
78     public void onClick(View v)
79     {
80         Log.d(TAG, "The onClick() HelloButton event");
81
82         helloTextView.setText("Hello World!");
83     }
84 };
85
86
87
88
89
90
91
92
93
94
95
```

Let's build Hello
World App

Questions?

How to help yourself?

18

- ▶ Use IDE autocompletion and code generation
- ▶ Compile your code often
- ▶ Change only one feature at a time
- ▶ Fix errors based on the error messages and IDE hints
 - ▶ Compile errors & runtime errors
- ▶ Use debugging & break points to step through the code
- ▶ Use logging statements to create informative logs
- ▶ Use try catch statements to handle errors elegantly
- ▶ Refer to documentation and tutorials...

How to use references?

19

- ▶ Make sure the reference is current
 - ▶ 3-5 years old is sometimes already obsolete
- ▶ Make sure the reference has authority
- ▶ Make sure the coding strategy is good
 - ▶ Not just “functional”
- ▶ Use the reference to understand the principles and write your own version to accomplish your specific goal

A few don'ts...

20

- ▶ DO NOT copy/paste code from any source other than your own project
- ▶ DO NOT start a project using an existing project if your goal is not explicitly stated to be “extending this existing project”
- ▶ DO NOT reuse whole source code files unless they are explicitly cited as the work of the original authors